ICTi – From Hype to Prototype

By: Samuel R. Tito Infantas

01

CreditLLM: Constructing Financial AI Assistant for Credit Products using Financial LLM and Few Data

Part I: The Relevant Signal

01 – The relevant Signal

Promise I

Create a Financial AI Assistant for credit products with few data.

Promise II

Propose a categorization framework for customer communication covering different task.

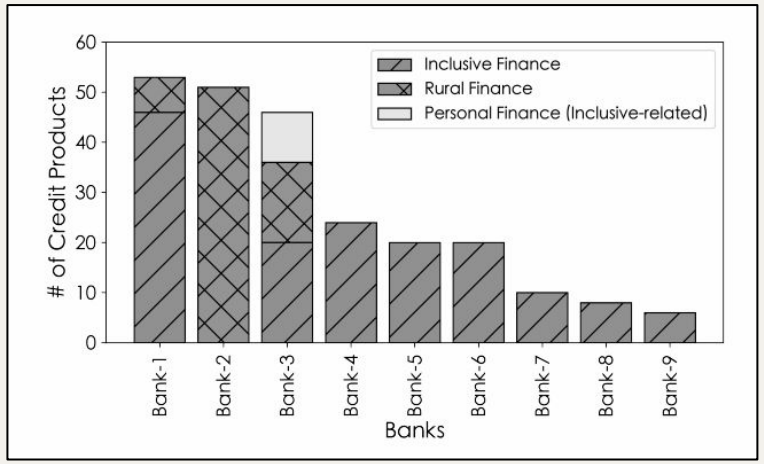
Promise III

Approach for develop a mount of synthetic data which will be used to develop the LLM loan-related ability

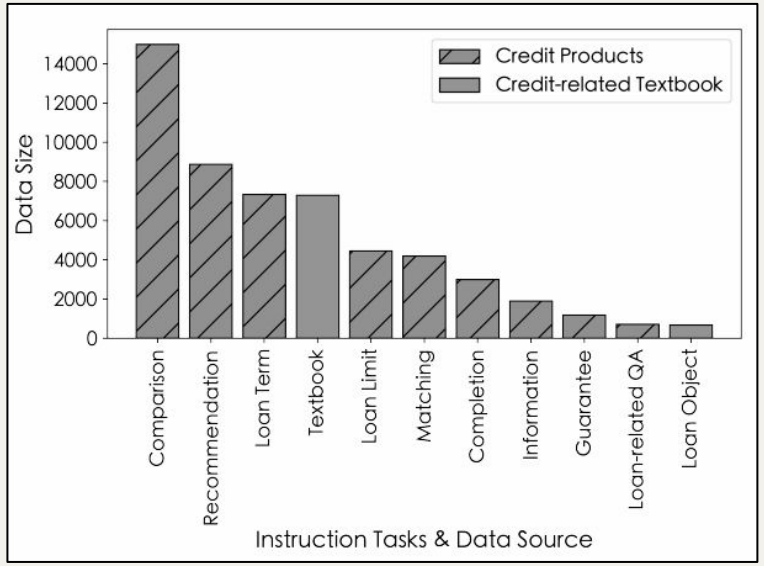
Promise IV

The feasibility to developing LLM with finance capability with few real-world data

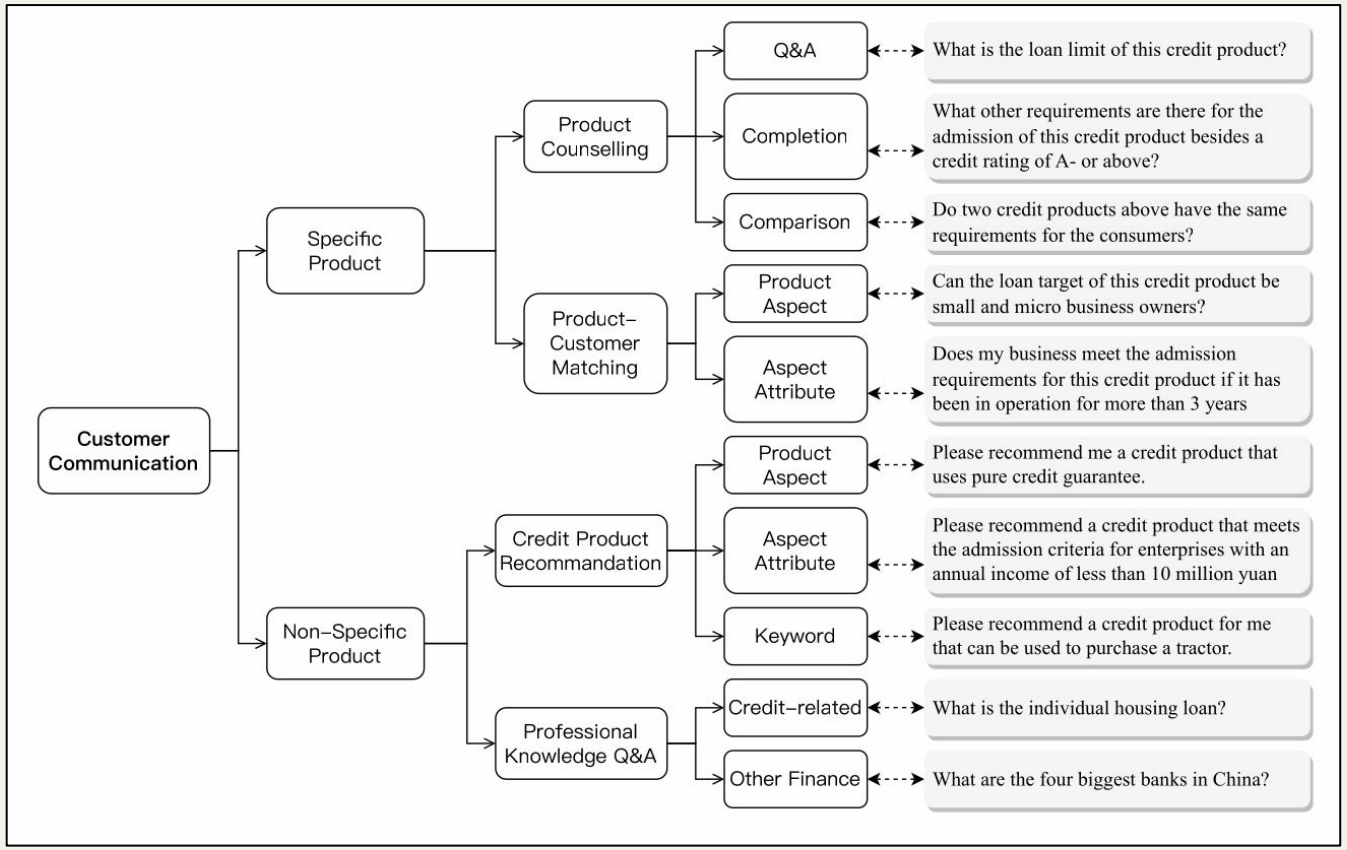
Paper Summary



Credit products collected

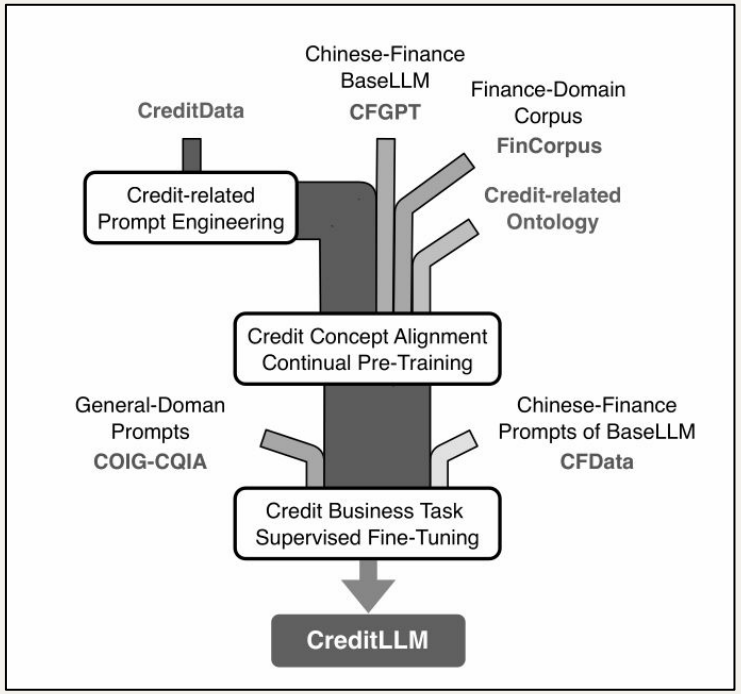


Synthetic Data

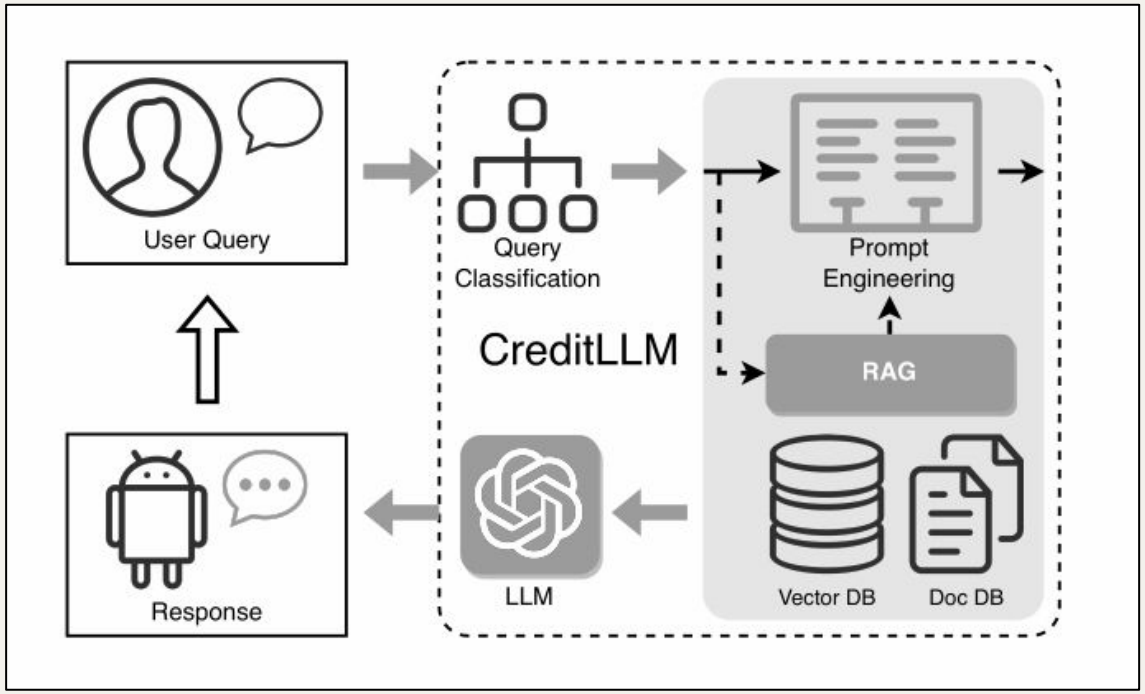


Hierarchical LLM communication future capability for communication with customers

Paper Summary



CPT & SFT process



Credit LLM Architecture

# of credit product	238
Avg. # of tokens per credit product	233
# of prompt data	52,751
Avg. # of prompts generated per credit product	222

Table 1: The data of credit products and prompts.

CPT Data of (CreditLLM / Base LLM)	1.65%
SFT Data of (CreditLLM / Base LLM)	5.28%
Information Gain of CreditLLM	155.45%

Table 4: The comparison of data used for CreditLLM and its Base LLM. "CPT" and "SFT" stand for "continual pre-training" and "supervised fine-tuning".

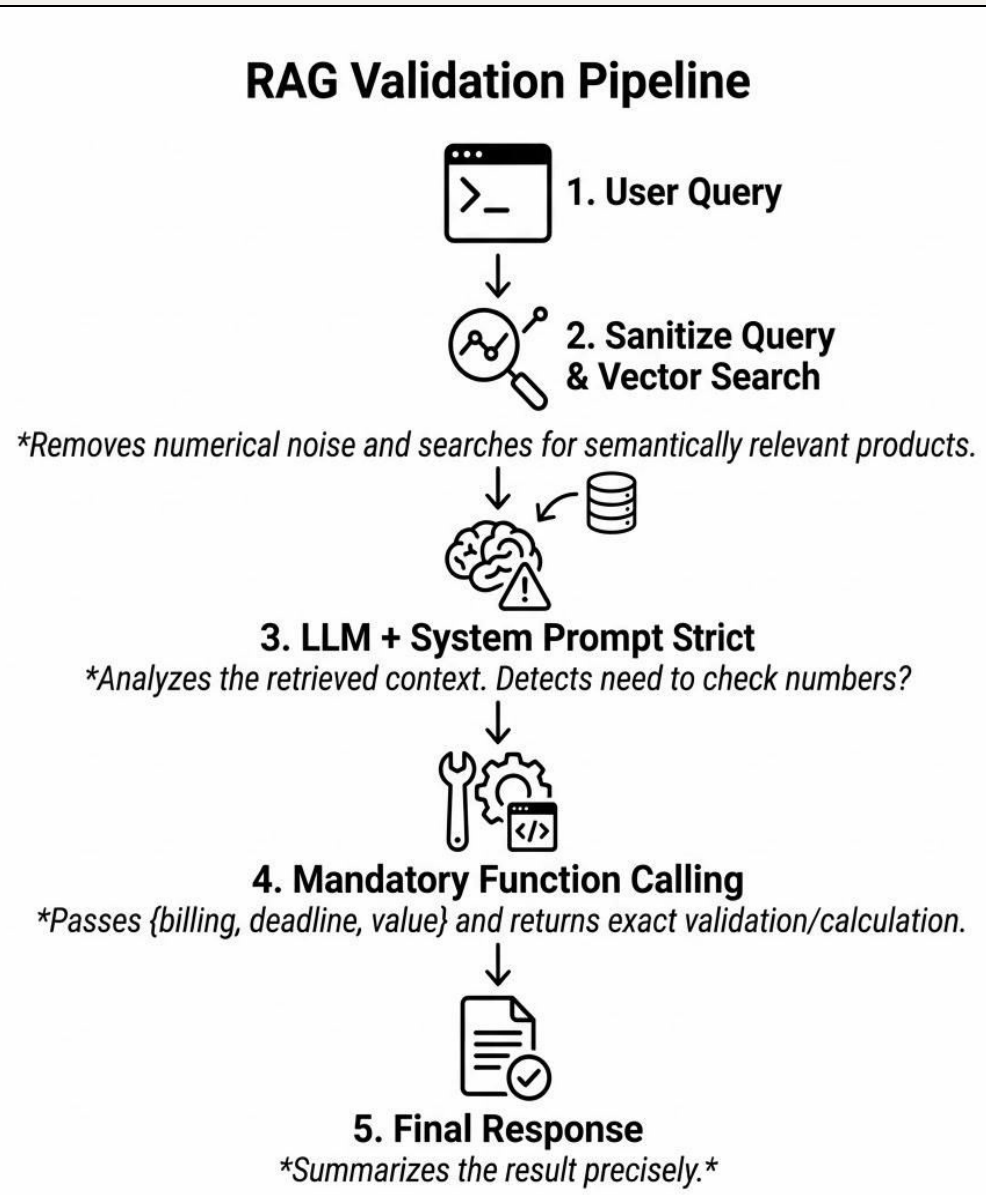
Opportunities, Risks & Boundaries

Sinal	Risk/Problem	Hype Illusion
It is easy and feasible to create a Financial AI Assistant for credit products with few data that allows to communicate with customer.	The fine-tuning approach introduces the risk of catastrophic forgetting. The amount of synthetic prompts (53K) x the credits products suggest a possible overfitting.	There is a core contradiction in the current approach: Fine-Tune x RAG? The promise to develop an Finance LLM with “few data” is subjective, since it relies heavily on synthetic data.

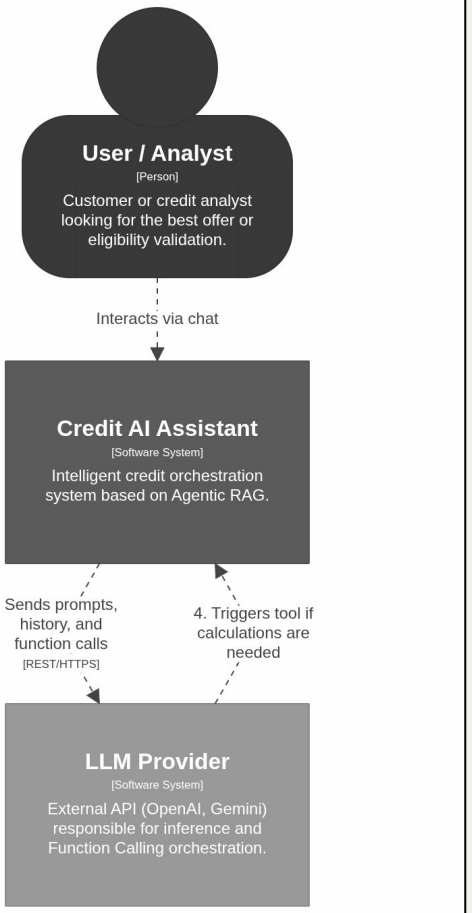
Prototype *Approach*

Instead of Fine-Tuning the model, the best approach to set up a Loop or pipeline like the paper approach.

It creates a scalable, improving and easily updated services.



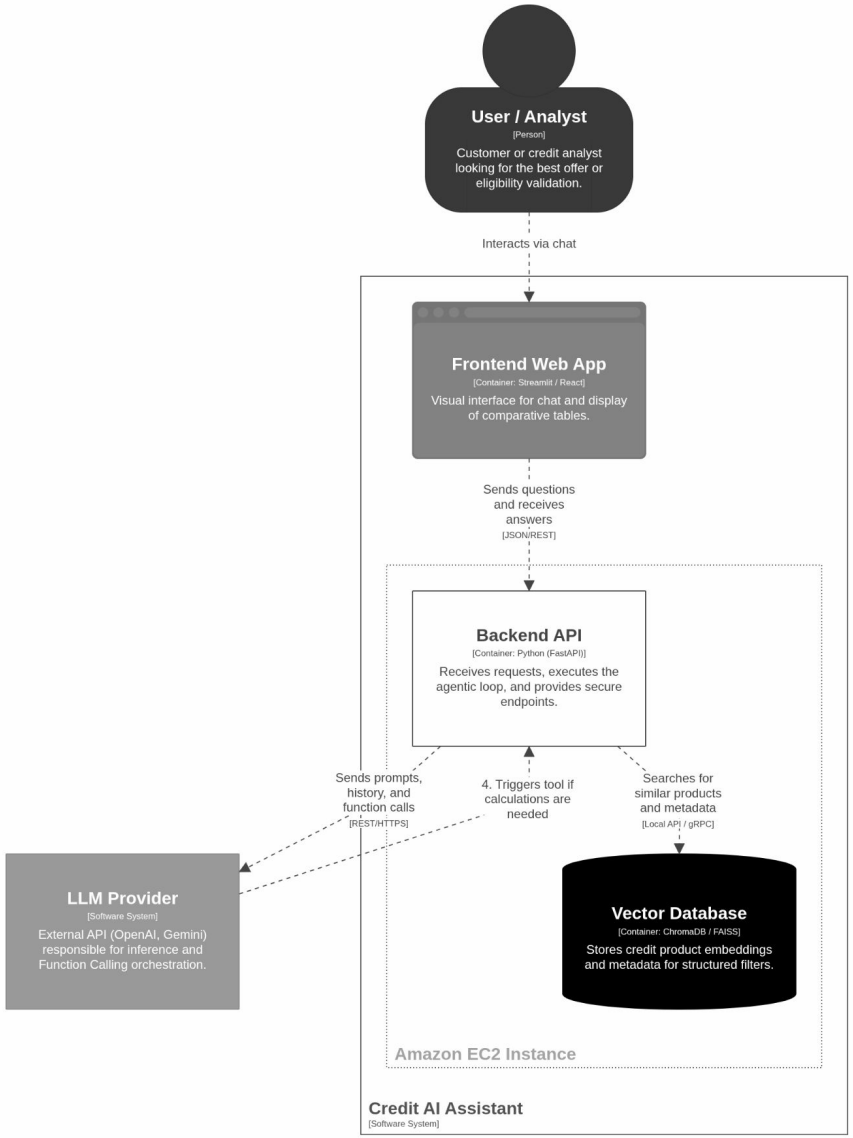
Architecture



System Context View: Credit AI Assistant

Context Diagram: The main system and its external integrations.

Wednesday, 5 August 2026 at 19:15 Brasilia Standard Time

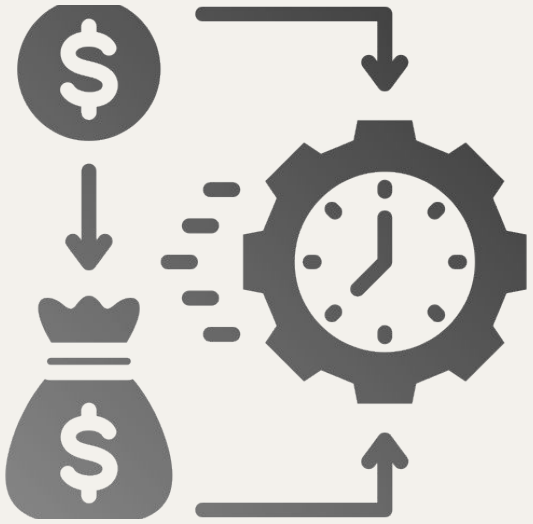


Container View: Credit AI Assistant

Container Diagram: The division between Frontend and Backend running on EC2.

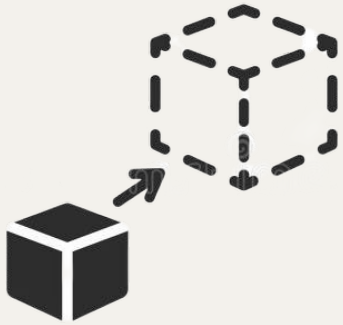
Wednesday, 5 August 2026 at 19:16 Brasilia Standard Time

Why that approach is better?



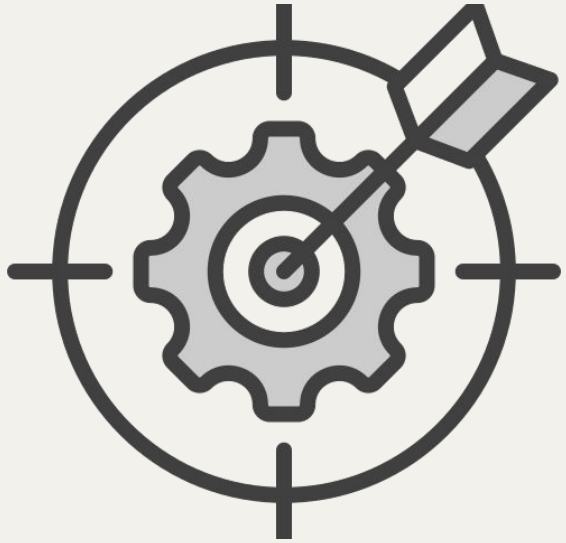
Cost Efficiency

The Agentic RAG solution is more faster and cheaper to implement.



Scalability & Uncouple

The ARS is LLM-Agnostic and makes data updates easier.



Accuracy

Using the tools such as Function Callings we force the calculus to be handled outside the LLM.

Thank you!